

# Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

**Nombres:** María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

**Fecha:** 4 de Mayo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada RESNET

## 1. Importe de Librerías

```
In [3]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import classification_report

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

```

In [4]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-05-11 20:05:07.318881

## 2. Definición de Rutas y Parámetros

```

In [6]: data_dir = 'Datos Finales Enfermedad Balanceados' # Reemplaza con la ruta a los datos
        metadata_file = 'archivos_datos_balanceados_enfermedad.csv' # Reemplaza con la ruta al archivo CSV
        image_height = 150
        image_width = 150
        batch_size = 15
        num_classes = None # Se determinará automáticamente del archivo CSV

```

```

In [7]: try:
        metadata = pd.read_csv(metadata_file, sep=";")
        metadata["Planta_Condición_Enfermedad"] = metadata["Planta_Estado_Español"]
        metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})

        # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de archivos) y 'label' (o el nombre de tu columna de etiquetas) existan.

        if 'image_id' not in metadata.columns or 'Planta_Condición_Enfermedad' not in metadata.columns:
            raise ValueError("El archivo CSV debe contener las columnas 'image_id' y 'Planta_Condición_Enfermedad'")
        class_names = metadata["Planta_Condición_Enfermedad"].unique().tolist()

```

```

num_classes = len(class_names)
print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file}'")
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()

```

Se encontraron 37 clases: ['Fresa\_Saludable\_Saludable', 'Coliflor\_Enferma\_Podredumbre blanda', 'Papa\_Enferma\_Quemado de la papa', 'Maíz\_Saludable\_Saludable', 'Tomate\_Enferma\_Pinta en ojo de buey', 'Papa\_Enferma\_Pudrición de la papa', 'Tomate\_Enferma\_Pudrición de la papa', 'Tomate\_Enferma\_Moteado del tomate', 'Yuca\_Enferma\_Venas marrones de la yuca', 'Tomate\_Enferma\_Hongo blanco del tomate', 'Coliflor\_Saludable\_Saludable', 'Manzana\_Enferma\_Hongo del manzano', 'Tomate\_Enferma\_Puntos negros del tomate', 'Yuca\_Enferma\_Pudrición de la yuca', 'Yuca\_Enferma\_Manchas verdes en la yuca', 'Tomate\_Enferma\_Quemado de la papa', 'Maíz\_Enferma\_Moho rojo del maíz', 'Pepino Cohombro\_Saludable\_Saludable', 'Tomate\_Saludable\_Saludable', 'Yuca\_Saludable\_Saludable', 'Coliflor\_Enferma\_Pudrición del fruto', 'Tomate\_Enferma\_Hojas enrolladas del tomate', 'Manzana\_Enferma\_Costra de la manzana', 'Maíz\_Enferma\_Quemazón del maíz', 'Fríjol\_Saludable\_Saludable', 'Tomate\_Enferma\_Polvillo rojo', 'Fríjol\_Enferma\_Mancha angular', 'Coliflor\_Enferma\_Moho azul', 'Tomate\_Enferma\_Puntitos marrones en las hojas', 'Maíz\_Enferma\_Pinta gris', 'Manzana\_Enferma\_Pudrición del fruto', 'Manzana\_Saludable\_Saludable', 'Fríjol\_Enferma\_Oxido de frijol', 'Pepino Cohombro\_Enferma\_Enferma', 'Fresa\_Enferma\_Hojas quemadas de la fresa', 'Papa\_Saludable\_Saludable', 'Yuca\_Enferma\_Manchas amarillas en la yuca']

```
In [8]: train, test = train_test_split(metadata, test_size=0.2, random_state=42, shuffle=True)
        train.shape, test.shape
```

```
Out[8]: ((8880, 12), (2220, 12))
```

```
In [9]: # Visualización del preprocesamiento y data augmentation
```

```

datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)

sample_df = train.sample(n=1)
sample_image_path = os.path.join(data_dir, sample_df.iloc[0]['image_id'])
sample_label = sample_df.iloc[0]['Planta_Condición_Enfermedad']

```

```

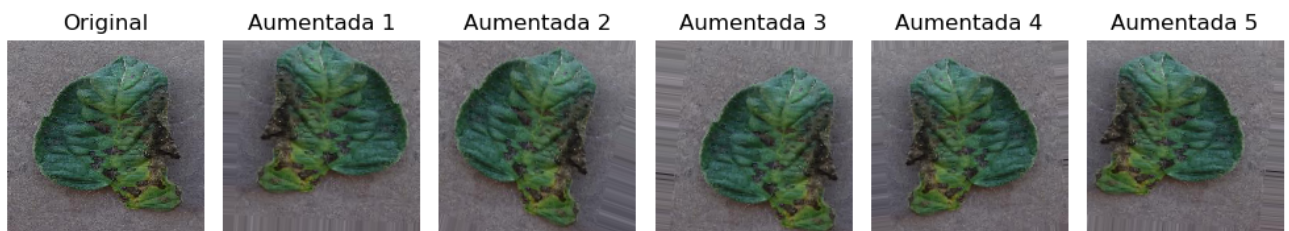
img = load_img(sample_image_path, target_size=(image_height, image_width))
x = img_to_array(img)
x = x.reshape((1,) + x.shape)

# Mostrar imagen original
plt.figure(figsize=(10, 5))
plt.subplot(1, 6, 1)
plt.imshow(img)
plt.title("Original")
plt.axis("off")

# Mostrar 5 imágenes aumentadas
i = 2
for batch in datagen.flow(x, batch_size=1):
    plt.subplot(1, 6, i)
    plt.imshow(batch[0])
    plt.title(f"Aumentada {i-1}")
    plt.axis("off")
    i += 1
    if i > 6:
        break
plt.suptitle(f"Ejemplo de aumentación para clase: {sample_label}", fontsize=
plt.tight_layout()
plt.show()

```

Ejemplo de aumentación para clase: Tomate\_Enferma\_Quemado de la papa



### 3. Cargar y procesar los metadatos

```
In [11]: train.Planta_Condición_Enfermedad.value_counts()
```

```

Out[11]: Planta_Condición_Enfermedad
Tomate_Enferma_Pinta en ojo de buey      240
Tomate_Saludable_Saludable                240
Maíz_Enferma_Quemazón del maíz            240
Tomate_Enferma_Moteado del tomate          240
Manzana_Enferma_Pudrición del fruto        240
Coliflor_Enferma_Moho azu                  240
Fresa_Saludable_Saludable                 240
Papa_Enferma_Pudrición de la papa          240
Tomate_Enferma_Hongo blanco del tomate     240
Papa_Enferma_Quemado de la papa            240
Yuca_Saludable_Saludable                  240
Fríjol_Enferma_Oxido de frijol             240
Pepino Cohombro_Saludable_Saludable        240
Tomate_Enferma_Polvillo rojo               240
Fríjol_Saludable_Saludable                240
Yuca_Enferma_Manchas verdes en la yuca     240
Tomate_Enferma_Puntitos marrones en las hojas 240
Tomate_Enferma_Puntos negros del tomate    240
Manzana_Enferma_Costra de la manzana       240
Pepino Cohombro_Enferma_Enferma           240
Tomate_Enferma_Pudrición de la papa        240
Fríjol_Enferma_Mancha angular              240
Coliflor_Saludable_Saludable              240
Maíz_Enferma_Pinta gris                    240
Papa_Saludable_Saludable                   240
Maíz_Saludable_Saludable                   240
Tomate_Enferma_Hojas enrolladas del tomate 240
Yuca_Enferma_Pudrición de la yuca          240
Maíz_Enferma_Moho rojo del maíz            240
Manzana_Saludable_Saludable               240
Tomate_Enferma_Quemado de la papa          240
Coliflor_Enferma_Podredumbre blanda        240
Manzana_Enferma_Hongo del manzano          240
Yuca_Enferma_Manchas amarillas en la yuca  240
Fresa_Enferma_Hojas quemadas de la fresa   240
Yuca_Enferma_Venas marrones de la yuca     240
Coliflor_Enferma_Pudrición del fruto       240
Name: count, dtype: int64

```

## 4. Crear un generador de datos

### 4.1. Sin Pre-processing Function

```

In [14]: train.Planta_Condición_Enfermedad = train.Planta_Condición_Enfermedad.astype(
datagen = ImageDataGenerator(

```

```

        rescale=1./255,
        rotation_range=20,
        width_shift_range=0.1,
        height_shift_range=0.1,
        shear_range=0.1,
        zoom_range=0.1,
        horizontal_flip=True,
        fill_mode='nearest'
    )

    train_generator = datagen.flow_from_dataframe(
        dataframe=train,
        directory=data_dir,
        x_col='image_id',
        y_col='Planta_Condición_Enfermedad',
        target_size=(image_height, image_width),
        batch_size=batch_size,
        class_mode='categorical', # Importante para clasificación multiclase
        subset='training',
        seed=42,
        classes=class_names # Asegurarse del orden de las clases
    )

    validation_generator = datagen.flow_from_dataframe(
        dataframe=train,
        directory=data_dir,
        x_col='image_id',
        y_col='Planta_Condición_Enfermedad',
        target_size=(image_height, image_width),
        batch_size=batch_size,
        class_mode='categorical',
        subset='validation',
        seed=42,
        classes=class_names
    )

```

Found 8879 validated image filenames belonging to 37 classes.

Found 0 validated image filenames belonging to 37 classes.

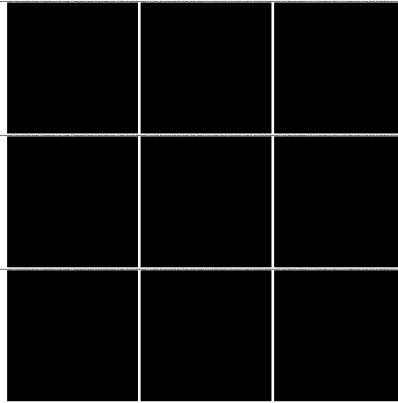
```

In [15]: # Obtener un lote de imágenes aumentadas
         images, labels = next(train_generator)

         # Mostrar las primeras 9 imágenes del lote
         plt.figure(figsize=(50, 50))
         for i in range(9):
             plt.subplot(3, 3, i + 1)
             plt.imshow(images[i].astype("uint8"))
             plt.title(f"Clase: {train_generator.class_indices}")
             plt.axis('off')
         plt.tight_layout()

```

```
plt.show()
```



## 4.2. Con Pre-processing Function

```
In [17]: def preprocess_leaf_image(img, target_size=(150, 150)):
# 1. Asegurar que la imagen esté en uint8
if img.dtype != np.uint8:
    img = np.clip(img * 255, 0, 255).astype(np.uint8)

# 2. Convertir a BGR y redimensionar
img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
img_resized = cv2.resize(img_bgr, target_size, interpolation=cv2.INTER_A

# 3. Convertir a HSV y crear máscara para verdes
hsv = cv2.cvtColor(img_resized, cv2.COLOR_BGR2HSV)
lower_green = np.array([25, 40, 40])
upper_green = np.array([85, 255, 255])
mask = cv2.inRange(hsv, lower_green, upper_green)

# 4. Limpiar máscara con operaciones morfológicas
kernel = np.ones((3, 3), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

# 5. Aplicar máscara si hay suficiente área verde
green_ratio = np.sum(mask > 0) / (mask.shape[0] * mask.shape[1])
if green_ratio >= 0.01:
    segmented = cv2.bitwise_and(img_resized, img_resized, mask=mask)
else:
    segmented = img_resized

# 6. Mejorar contraste usando CLAHE (en espacio LAB)
lab = cv2.cvtColor(segmented, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
l_clahe = clahe.apply(l)
lab_clahe = cv2.merge((l_clahe, a, b))
img_clahe = cv2.cvtColor(lab_clahe, cv2.COLOR_LAB2BGR)
```

```
# 7. Aplicar filtro bilateral para reducir ruido preservando bordes
img_filtered = cv2.bilateralFilter(img_clahe, d=9, sigmaColor=75, sigmaS

# 8. Convertir de vuelta a RGB y normalizar
img_rgb = cv2.cvtColor(img_filtered, cv2.COLOR_BGR2RGB)
return img_rgb.astype(np.float32) / 255.0
```

```
In [18]: # Cargar una imagen de ejemplo
img_path = "/Users/mfia/Documents/MFIA - PUJ - Maestría/Proyecto de Grado/0.
original_img = load_img(img_path)
original_array = img_to_array(original_img) / 255.0

# Aplicar preprocesamiento
processed_img = preprocess_leaf_image(original_array)

# Visualizar resultado
fig, axes = plt.subplots(1, 2, figsize=(5, 2))
axes[0].imshow(original_array)
axes[0].set_title("Original")
axes[0].axis("off")

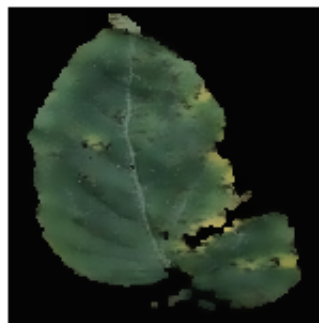
axes[1].imshow(processed_img)
axes[1].set_title("Preprocesada")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```

Original



Preprocesada



```
In [19]: import random
def generar_numero_aleatorio(minimo, maximo):
    return random.randint(minimo, maximo)

# Ejemplo de uso:
min_valor = 1
max_valor = 7000
numero_aleatorio = generar_numero_aleatorio(min_valor, max_valor)
print(f"Número aleatorio entre {min_valor} y {max_valor}: {numero_aleatorio}")
```



```
# Obtener la ruta de la imagen desde el DataFrame
image_filename = train.iloc[numero_aleatorio]['image_id']
image_path = os.path.join(data_dir, image_filename)

# Cargar la imagen original
original_img = load_img(image_path) # Carga como objeto PIL
original_array = img_to_array(original_img) / 255.0 # Normaliza para imshow

# Visualizar
plt.figure(figsize=(6, 6))
plt.imshow(original_array)
plt.axis("off")
plt.title("Imagen original sin preprocesamiento")
plt.show()
```

Número aleatorio entre 1 y 7000: 1275

Imagen original sin preprocesamiento



```
In [20]: train.Planta_Condición_Enfermedad = train.Planta_Condición_Enfermedad.astype('str')
          datagen = ImageDataGenerator(
```

```

validation_split = 0.2,
preprocessing_function = preprocess_leaf_image,
rescale=1./255,
rotation_range=20,
width_shift_range=0.1,
height_shift_range=0.1,
shear_range=0.1,
zoom_range=0.1,
horizontal_flip=True,
fill_mode='nearest'
)

train_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Condición_Enfermedad',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical', # Importante para clasificación multiclase
    subset='training',
    seed=42,
    classes=class_names # Asegurarse del orden de las clases
)

validation_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Condición_Enfermedad',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=42,
    classes=class_names
)

```

Found 7104 validated image filenames belonging to 37 classes.

Found 1775 validated image filenames belonging to 37 classes.

```

In [21]: generator = datagen.flow_from_dataframe(train.iloc[numero_aleatorio:numero_a
        directory=data_dir,
        x_col='image_id',
        y_col='Planta_Condición_Enfermedad',
        target_size=(image_height, image_wic
        batch_size=batch_size,
        class_mode='categorical')

# Obtener un lote (X_batch, y_batch) del generador

```

```

images, labels = next(generator)

# Mapa de índices a nombres de clase
class_indices = generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}

# Visualizar la imagen
plt.figure(figsize=(4, 4))
plt.imshow(images[0])
plt.axis('off')

# Obtener nombre de la clase (one-hot a texto)
predicted_class = index_to_class[np.argmax(labels[0])]
plt.title(f"Clase: {predicted_class}")
plt.show()
print("Min:", images[0].min(), "Max:", images[0].max())

```

Found 1 validated image filenames belonging to 1 classes.

Clase: Fríjol\_Saludable\_Saludable



Min: 0.0 Max: 0.003921569

## 5. Cargar el modelo ResNet50 preentrenado (sin la capa superior de clasificación)

```
In [23]: base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(in
```

## 6. Congelar las capas convolucionales base

```
In [25]: for layer in base_model.layers:
          layer.trainable = False
```

## 7. Añadir una nueva capa de clasificación personalizada

```
In [27]: x = base_model.output
x = GlobalAveragePooling2D()(x)
# Puedes añadir más capas densas aquí si lo deseas
# x = Dense(1024, activation='relu')(x)
predictions = Dense(num_classes, activation='softmax')(x)
```

## 8. Crear el modelo final

```
In [29]: model = Model(inputs=base_model.input, outputs=predictions)
```

## 9. Compilar el modelo para fine-tuning de la última capa

```
In [31]: model.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])
```

## 10. Entrenar el modelo con la última capa descongelada

```
In [33]: epochs_fine_tune = 2 # Define el número de épocas para el fine-tuning
```

```
In [34]: print("Comenzando el entrenamiento de la última capa...")
history_fine_tune = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=epochs_fine_tune,
    verbose = 1
)
```

Comenzando el entrenamiento de la última capa...

Epoch 1/2

**473/473**  **179s** 375ms/step - accuracy: 0.0249 - loss: 3.6775 - val\_accuracy: 0.0277 - val\_loss: 3.6521

Epoch 2/2

**473/473**  **46s** 97ms/step - accuracy: 0.0000e+00 - loss: 3.7462 - val\_accuracy: 0.0277 - val\_loss: 3.6496

```
In [35]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-05-11 20:08:57.720378

Duración: 0:03:50.401497

## 11. Opcional: Descongelar algunas de las últimas capas del modelo base para un fine-tuning más profundo

```
In [37]: # unfreeze_layers = 5 # Número de capas a descongelar (contando desde la última)

# for layer in base_model.layers[-unfreeze_layers:]:
#     layer.trainable = True
```

## 12. Opcional: Recompilar el modelo después de descongelar las capas

```
In [39]: #model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # Usar Adam
#                    loss='categorical_crossentropy',
#                    metrics=['accuracy'])
```

## 13. Opcional: Entrenar el modelo con las capas descongeladas

```
In [41]: #epochs_deep_fine_tune = 5 # Define el número de épocas para el fine-tuning

#print(f"\nComenzando el fine-tuning profundo de las últimas {unfreeze_layers} capas")
#history_deep_fine_tune = model.fit(
#    train_generator,
#    steps_per_epoch=train_generator.samples // train_generator.batch_size,
#    validation_data=validation_generator,
#    validation_steps=validation_generator.samples // validation_generator.batch_size)
```

```
#     epochs=epochs_deep_fine_tune
#)
```

## 14. Evaluar el modelo final

```
In [43]: print("\nEvaluando el modelo...")
evaluation_result = model.evaluate(validation_generator)
print(f"Pérdida en la validación: {evaluation_result[0]:.4f}")
print(f"Precisión en la validación: {evaluation_result[1]:.4f}")
```

Evaluando el modelo...

**119/119**  **35s** 293ms/step - accuracy: 0.0293 - loss: 3.665

3

Pérdida en la validación: 3.6508

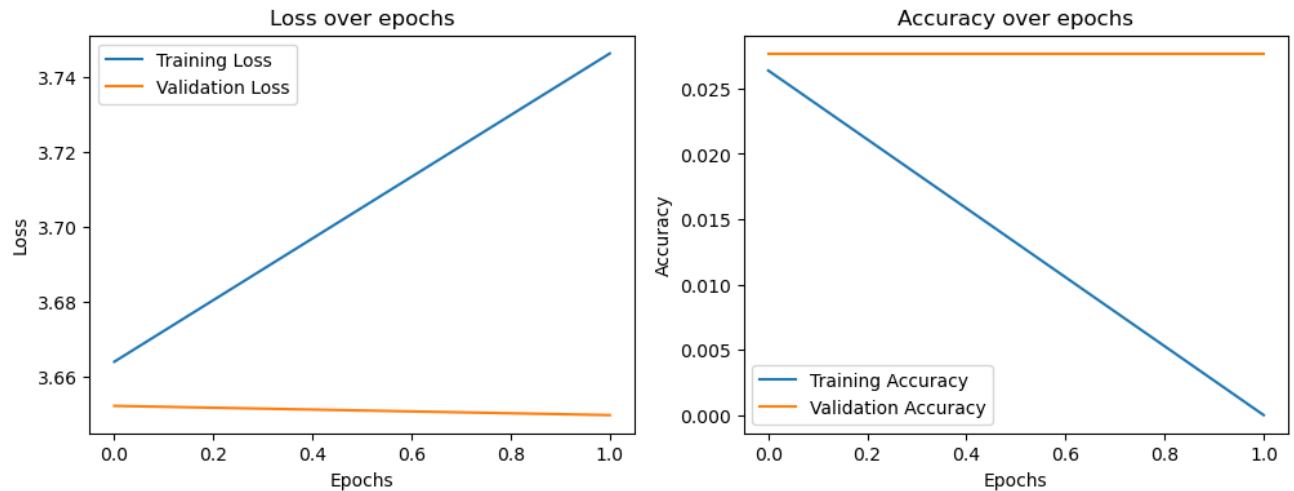
Precisión en la validación: 0.0276

```
In [44]: # Acceder a los datos de pérdida y precisión
loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
accuracy = history_fine_tune.history['accuracy']
val_accuracy = history_fine_tune.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_fine_tune.history['loss'], label='Training Loss')
plt.plot(history_fine_tune.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history_fine_tune.history['accuracy'], label='Training Accuracy')
plt.plot(history_fine_tune.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

Out[44]: <matplotlib.legend.Legend at 0x169f28890>



```
In [45]: epochs = list(range(1, 3))
data = {
    'Epoch': epochs,
    'Trainning Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

|   | Epoch | Trainning Loss | Validation Loss | Training Accuracy \ |
|---|-------|----------------|-----------------|---------------------|
| 0 | 1     | 3.663855       | 3.652054        | 0.026379            |
| 1 | 2     | 3.746219       | 3.649586        | 0.000000            |

|   | Validation Accuracy |
|---|---------------------|
| 0 | 0.027684            |
| 1 | 0.027684            |

```
In [46]: # Accuracy final en entrenamiento
val_loss = history_fine_tune.history['val_loss'][-1]
```

```
In [47]: print('Perdida final en Train:', history_fine_tune.history['loss'][-1])
print("Pérdida final en Validation:", history_fine_tune.history['val_loss'][-1])

print("Accuracy final en Train:", history_fine_tune.history['accuracy'][-1])
print("Accuracy final en Validation:", history_fine_tune.history['val_accuracy'][-1])
```

```
Perdida final en Train: 3.7462189197540283
Pérdida final en Validation: 3.6495859622955322
Accuracy final en Train: 0.0
Accuracy final en Validation: 0.027683615684509277
```



```
In [48]: class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo11_balanceado_enfermedad.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Fresa_Saludable_Saludable', 1: 'Coliflor_Enferma_Podredumbre blanda',
2: 'Papa_Enferma_Quemado de la papa', 3: 'Maíz_Saludable_Saludable', 4: 'Tomate_Enferma_Pinta en ojo de buey', 5: 'Papa_Enferma_Pudrición de la papa',
6: 'Tomate_Enferma_Pudrición de la papa', 7: 'Tomate_Enferma_Moteado del tomate', 8: 'Yuca_Enferma_Venas marrones de la yuca', 9: 'Tomate_Enferma_Hongo blanco del tomate', 10: 'Coliflor_Saludable_Saludable', 11: 'Manzana_Enferma_Hongo del manzano', 12: 'Tomate_Enferma_Puntos negros del tomate', 13: 'Yuca_Enferma_Pudrición de la yuca', 14: 'Yuca_Enferma_Manchas verdes en la yuca', 15: 'Tomate_Enferma_Quemado de la papa', 16: 'Maíz_Enferma_Moho rojo del maíz', 17: 'Pepino Cohombro_Saludable_Saludable', 18: 'Tomate_Saludable_Saludable', 19: 'Yuca_Saludable_Saludable', 20: 'Coliflor_Enferma_Pudrición del fruto', 21: 'Tomate_Enferma_Hojas enrolladas del tomate', 22: 'Manzana_Enferma_Costra de la manzana', 23: 'Maíz_Enferma_Quemazón del maíz', 24: 'Fríjol_Saludable_Saludable', 25: 'Tomate_Enferma_Polvillo rojo', 26: 'Fríjol_Enferma_Mancha angular', 27: 'Coliflor_Enferma_Moho azul', 28: 'Tomate_Enferma_Puntitos marrones en las hojas', 29: 'Maíz_Enferma_Pinta gris', 30: 'Manzana_Enferma_Pudrición del fruto', 31: 'Manzana_Saludable_Saludable', 32: 'Fríjol_Enferma_Oxido de frijol', 33: 'Pepino Cohombro_Enferma_Enferma', 34: 'Fresa_Enferma_Hojas quemadas de la fresa', 35: 'Papa_Saludable_Saludable', 36: 'Yuca_Enferma_Manchas amarillas en la yuca'}
```

## 15. Testeo del Modelo

```
In [50]: test.Planta_Condición_Enfermedad = test.Planta_Condición_Enfermedad.astype('float32')

datagen = ImageDataGenerator(
    validation_split = 0.2,
    preprocessing_function = preprocess_leaf_image,
    rotation_range = 45,
    zoom_range = 0.2,
    horizontal_flip = True,
    vertical_flip = True,
    fill_mode = 'nearest',
    shear_range = 0.1,
    height_shift_range = 0.1,
    width_shift_range = 0.1
)

test_generator = datagen.flow_from_dataframe(test,
                                             directory = "Datos Finales",
                                             subset = "training",
                                             x_col = "image_id",
                                             y_col = "Planta_Condición_Enfermedad",
```



```
target_size = (image_height, image_width),
batch_size = batch_size,
class_mode = "categorical")
```

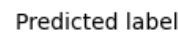
Found 1654 validated image filenames belonging to 37 classes.

```
In [51]: # -----
# 1. Obtener predicciones completas para test
# -----
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes
```

111/111  36s 314ms/step

```
In [52]: # -----
# 2. Matriz de Confusión
# -----
plt.figure(figsize=(200, 80))
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener
disp.plot(cmap="Blues", xticks_rotation=90)
plt.title("Matriz de Confusión - Test")
plt.savefig("matriz_confusion_test_modelo_11.png", dpi=300, bbox_inches='tight')
plt.show()
```

<Figure size 20000x8000 with 0 Axes>



Test Accuracy: 0.0151

```
# Obtener nombres de clases
```

```
# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=0.95)
plt.show()
```

1/1 ————— 1s 768ms/step

```

-----
ValueError                                Traceback (most recent call last)
Cell In[54], line 28
    26 plt.figure(figsize=(15, 8))
    27 for i in range(num_samples):
--> 28     plt.subplot(3, 4, i + 1)
    29     img_to_show = normalize_for_display(x_batch[i])
    30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs)))
    ):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13

```



```
In [56]: # 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

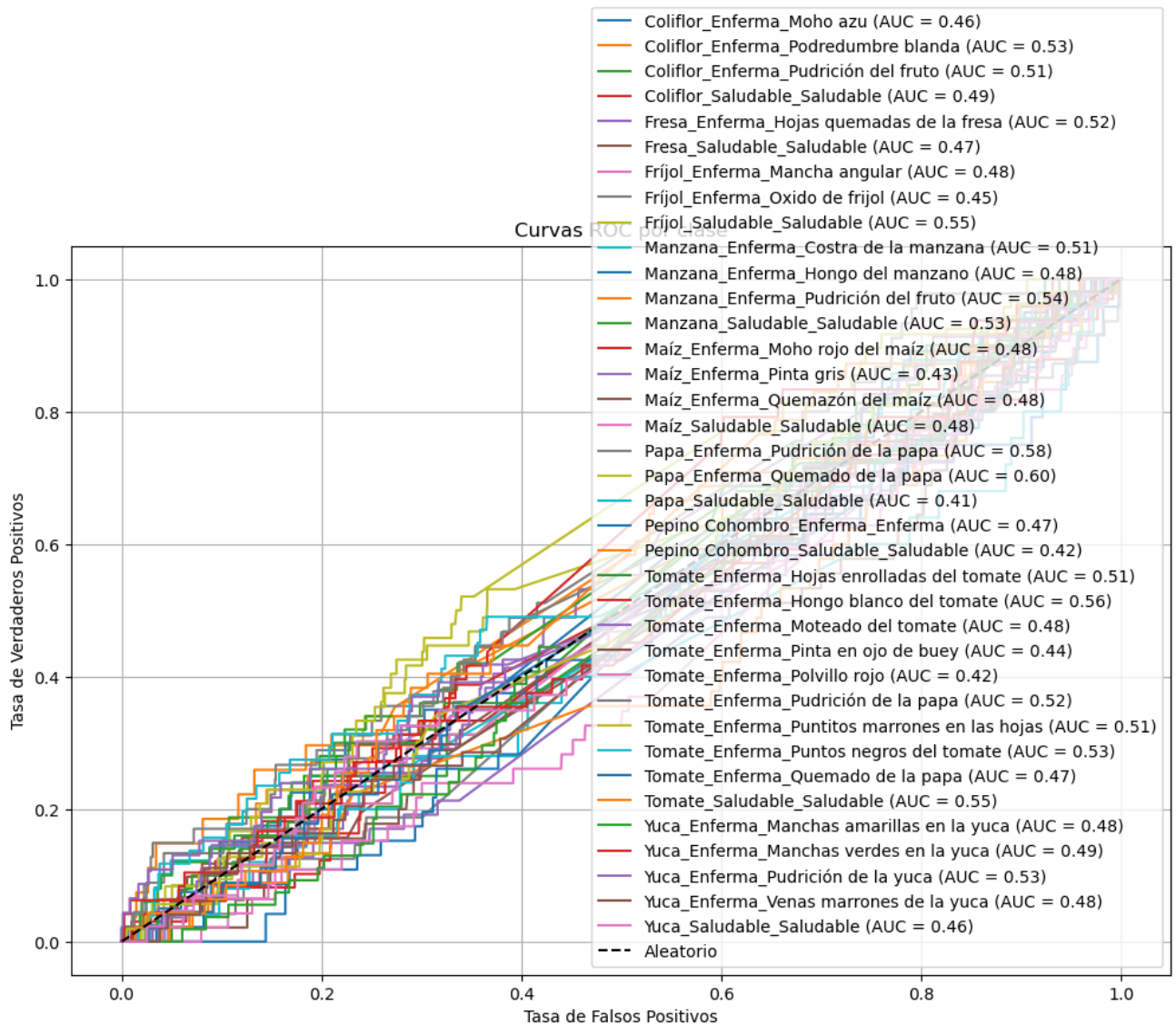
# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



```
In [58]: from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

AUC Macro promedio: 0.4951

AUC Micro promedio: 0.4929

```
In [60]: # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes
```

```

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))

```

Accuracy: 0.0151  
 F1 Score (macro): 0.0008  
 F1 Score (micro): 0.0151  
 F1 Score (weighted): 0.0005

Reporte de clasificación por clase:

|         |                                          | precision | recall | f1-score |
|---------|------------------------------------------|-----------|--------|----------|
| support |                                          |           |        |          |
|         | Coliflor_Enferma_Moho azu                | 0.00      | 0.00   | 0.00     |
| 24      |                                          |           |        |          |
|         | Coliflor_Enferma_Podredumbre blanda      | 0.00      | 0.00   | 0.00     |
| 27      |                                          |           |        |          |
|         | Coliflor_Enferma_Pudrición del fruto     | 0.00      | 0.00   | 0.00     |
| 16      |                                          |           |        |          |
|         | Coliflor_Saludable_Saludable             | 0.00      | 0.00   | 0.00     |
| 33      |                                          |           |        |          |
|         | Fresa_Enferma_Hojas quemadas de la fresa | 0.00      | 0.00   | 0.00     |
| 47      |                                          |           |        |          |
|         | Fresa_Saludable_Saludable                | 0.00      | 0.00   | 0.00     |
| 51      |                                          |           |        |          |
|         | Fríjol_Enferma_Mancha angular            | 0.00      | 0.00   | 0.00     |
| 48      |                                          |           |        |          |
|         | Fríjol_Enferma_Oxido de frijol           | 0.00      | 0.00   | 0.00     |
| 48      |                                          |           |        |          |
|         | Fríjol_Saludable_Saludable               | 0.00      | 0.00   | 0.00     |
| 47      |                                          |           |        |          |
|         | Manzana_Enferma_Costra de la manzana     | 0.00      | 0.00   | 0.00     |
| 49      |                                          |           |        |          |
|         | Manzana_Enferma_Hongo del manzano        | 0.00      | 0.00   | 0.00     |
| 40      |                                          |           |        |          |
|         | Manzana_Enferma_Pudrición del fruto      | 0.00      | 0.00   | 0.00     |

|    |                                               |      |      |      |
|----|-----------------------------------------------|------|------|------|
| 48 | Manzana_Saludable_Saludable                   | 0.00 | 0.00 | 0.00 |
| 50 | Maíz_Enferma_Moho rojo del maíz               | 0.00 | 0.00 | 0.00 |
| 49 | Maíz_Enferma_Pinta gris                       | 0.00 | 0.00 | 0.00 |
| 47 | Maíz_Enferma_Quemazón del maíz                | 0.00 | 0.00 | 0.00 |
| 47 | Maíz_Saludable_Saludable                      | 0.00 | 0.00 | 0.00 |
| 47 | Papa_Enferma_Pudrición de la papa             | 0.00 | 0.00 | 0.00 |
| 45 | Papa_Enferma_Quemado de la papa               | 0.00 | 0.00 | 0.00 |
| 48 | Papa_Saludable_Saludable                      | 0.02 | 1.00 | 0.03 |
| 25 | Pepino Cohombro_Enferma_Enferma               | 0.00 | 0.00 | 0.00 |
| 46 | Pepino Cohombro_Saludable_Saludable           | 0.00 | 0.00 | 0.00 |
| 45 | Tomate_Enferma_Hojas enrolladas del tomate    | 0.00 | 0.00 | 0.00 |
| 50 | Tomate_Enferma_Hongo blanco del tomate        | 0.00 | 0.00 | 0.00 |
| 48 | Tomate_Enferma_Moteado del tomate             | 0.00 | 0.00 | 0.00 |
| 55 | Tomate_Enferma_Pinta en ojo de buey           | 0.00 | 0.00 | 0.00 |
| 45 | Tomate_Enferma_Polvillo rojo                  | 0.00 | 0.00 | 0.00 |
| 46 | Tomate_Enferma_Pudrición de la papa           | 0.00 | 0.00 | 0.00 |
| 47 | Tomate_Enferma_Puntitos marrones en las hojas | 0.00 | 0.00 | 0.00 |
| 53 | Tomate_Enferma_Puntos negros del tomate       | 0.00 | 0.00 | 0.00 |
| 51 | Tomate_Enferma_Quemado de la papa             | 0.00 | 0.00 | 0.00 |
| 45 | Tomate_Saludable_Saludable                    | 0.00 | 0.00 | 0.00 |
| 47 | Yuca_Enferma_Manchas amarillas en la yuca     | 0.00 | 0.00 | 0.00 |
| 54 | Yuca_Enferma_Manchas verdes en la yuca        | 0.00 | 0.00 | 0.00 |
| 48 | Yuca_Enferma_Pudrición de la yuca             | 0.00 | 0.00 | 0.00 |
| 46 | Yuca_Enferma_Venas marrones de la yuca        | 0.00 | 0.00 | 0.00 |
| 49 | Yuca_Saludable_Saludable                      | 0.00 | 0.00 | 0.00 |



43

|      |              |      |      |      |
|------|--------------|------|------|------|
|      | accuracy     |      |      | 0.02 |
| 1654 |              |      |      |      |
|      | macro avg    | 0.00 | 0.03 | 0.00 |
| 1654 |              |      |      |      |
|      | weighted avg | 0.00 | 0.02 | 0.00 |
| 1654 |              |      |      |      |

## 16. Guardar el modelo entrenado

In [63]: `model.save('modelo_12_resnet50_fine_tuned_balanceado_enfermedades_mejorado_12')`

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

In [64]: `class_indices = train_generator.class_indices # o test_generator.class_indices  
index_to_class = {v: k for k, v in class_indices.items()}`

In [ ]:

In [ ]: